ANYBOT PLATFORM | BRAINSWARM ROBOTICS

Practical Session 1

Platform overview, the terminal, and your first ROS 2 package

Audience	Undergraduate students, first practical session
Duration	Approximately 2 hours
Robot	Not required. All work is done in a preview session.
Outcome	A working workspace and a package whose script runs with ros2 run

1. Purpose of this document

This handout records what we covered in the first practical session so that you can repeat every step on your own, and so that anyone who missed the session can catch up before the second one. Work through it at a terminal rather than reading it passively. Everything here runs inside your AnyBot workspace in the browser, with nothing to install on your own machine.

2. The AnyBot platform in brief

AnyBot gives each student an isolated Linux container with ROS 2 Humble already installed, reachable entirely through a web browser. You log in, and the platform gives you a code editor, a terminal, and when you have booked one, a connection to a real robot in our laboratory.

The two kinds of session

Session type	What it gives you
Preview session	Always available. Your own container, editor, terminal and ROS 2 environment for writing and building code. It is not connected to any physical robot.
Booked session	Available during a slot you reserve. The same workspace, plus a live connection to the physical robot assigned to that booking.

Your files live in your personal workspace and persist between sessions. Closing the browser does not delete your work.

Finding your way around

- The editor pane on the left is where you create and edit files.
- The terminal is where you build and run them. Open a new terminal from the editor menu.
- The learning path panel holds the written material for each module.
- Your home directory inside the container is /home/user.

3. Working at the terminal

Almost everything in ROS 2 is driven from a terminal, so these few commands matter more than they first appear. Type them rather than copying them, at least the first time.

Command	What it does
pwd	Prints the directory you are currently in.
ls	Lists the contents of the current directory.
cd name	Moves into the directory called name.
cd ..	Moves up one level.
mkdir name	Creates a new directory.
nano file	Opens a file in a terminal text editor.
chmod +x file	Marks a file as executable.

> **Tab completion** Press the Tab key part way through a path or command and the shell finishes it for you. This is the fastest way to avoid typing mistakes in long paths.

4. Creating a ROS 2 workspace

A workspace is simply a directory with a src subdirectory inside it. Everything you write goes in src, and the build system creates the other directories for you.

```
cd ~
mkdir -p ros2_ws/src
cd ros2_ws
colcon build
```

After the first build you will see three new directories next to src:

- **build** holds intermediate files produced while compiling.
- **install** holds the finished result, and is the one ROS 2 actually reads.
- **log** holds the record of each build, which is where to look when one fails.

Every new terminal must be told where your workspace is. This is the single most common reason a command that worked a minute ago suddenly reports that nothing was found:

```
source /opt/ros/humble/setup.bash # the ROS 2 installation itself
source ~/ros2_ws/install/setup.bash # your own workspace, after building
```

> **Order matters** Source the ROS 2 installation first and your workspace second. Sourcing your workspace before it has ever been built will report a missing file, which is expected until the first successful colcon build.

5. Creating a package

A workspace holds packages, and a package is the unit ROS 2 understands: it has a name, a declared list of dependencies, and a build type. We created ours with the built in generator rather than by hand.

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake my_first_package
```

The generator produces a directory containing two files that control everything else:

File	What it controls
package.xml	The package name, version, maintainer, and the list of other packages this one depends on.
CMakeLists.txt	How the package is built and, importantly, what gets installed so that ros2 run can find it.

6. Writing a Python node and running it directly

We wrote a small Python file inside the package and ran it the ordinary way first, before involving ROS 2 tooling at all. This is a deliberate order: it separates problems in your code from problems in your build configuration.

```
cd ~/ros2_ws/src/my_first_package
mkdir -p scripts
nano scripts/hello_node.py
```

The file began with a shebang line, which tells the system which interpreter to use when the file is executed directly:

```
#!/usr/bin/env python3


def main():
print('Hello from my first package')


if __name__ == '__main__':
main()
```

We then marked the file executable and ran it with the interpreter named explicitly:

```
chmod +x scripts/hello_node.py
python3 scripts/hello_node.py
```

At this point the script works, but only because we pointed python3 straight at it. ROS 2 knows nothing about it yet. Running ros2 run at this stage reports that no executable was found, and that is the correct behaviour, not a fault.

7. Making the script run with ros2 run

For ros2 run to find a program, the build must install it into the package library directory. That is what the install block in CMakeLists.txt does. We opened the file and added the block below, above the final ament_package() line.

```
nano ~/ros2_ws/src/my_first_package/CMakeLists.txt
```

```
install(PROGRAMS
scripts/hello_node.py
DESTINATION lib/${PROJECT_NAME}
)
```

Then rebuild, source the workspace again, and run it the ROS 2 way:

```
cd ~/ros2_ws
colcon build --packages-select my_first_package
source install/setup.bash
ros2 run my_first_package hello_node.py
```

> **Why this step exists** install(PROGRAMS ...) is for scripts that are already executable, such as Python files. install(TARGETS ...) is the equivalent for compiled C++ programs, and you will use that form in the second session. Forgetting either one is the usual cause of a build that succeeds while ros2 run still finds nothing.

8. Checklist before the next session

You should be able to do each of the following without looking anything up. If one of them is shaky, repeat section 4 through 7 before the second session, since everything there builds directly on these.

1. Create a workspace and build it.
2. Source the ROS 2 installation and your own workspace in a new terminal.
3. Create a package with ament_cmake as its build type.
4. Write a Python file, make it executable, and run it with python3.
5. Add an install block to CMakeLists.txt so the same file runs with ros2 run.
6. Explain, in one sentence, the difference between building and installing.

9. What the second session adds

The first session was entirely offline: nothing you wrote touched a robot. The second session changes that. You will each book a slot on a real manipulator arm, read its state while it is running, and send it commands from code you have written yourself. The separate handout for Practical Session 2 covers that work in full.